

08 — Heaps & Priority Queues

DSA in Python — Interview Prep

Heaps & Priority Queues — Tutorial

Covers the 10 problems solved in this folder. A heap answers exactly one question efficiently, repeatedly: "what's the smallest (or largest) item in this collection right now?" — in $O(\log n)$ per insert/remove instead of the $O(n)$ a plain list scan would need, and without the overhead of keeping the whole collection fully sorted. Python's `heapq` only implements a *min*-heap, so most of this folder's problems begin with the same trick: negate values on the way in and out to simulate a max-heap.

The patterns

1. Direct simulation (01)

`last_stone_weight` is the most literal use of a heap: the problem statement itself says "repeatedly take the two largest," so a max-heap (via negation) *is* the algorithm, not just an optimization of one. Worth doing first specifically because the heap maps onto the problem with zero translation needed.

2. Size-k heap for top-k extraction (02, 03, 04)

The single most common heap pattern in interviews. Maintain a heap of exactly size k holding the "best k so far"; its root is the *worst* of those k , making it the one to evict the instant something better appears. This turns an $O(n \log n)$ full sort into $O(n \log k)$ — a real win whenever k is much smaller than n .

- `kth_largest_element_in_array`: min-heap of size k ; root is directly the k th largest (the smallest among the top k).
- `kth_largest_element_in_a_stream`: identical invariant, but the heap persists across calls instead of being rebuilt each time — the "streaming" version of the same idea.
- `k_closest_points_to_origin`: same invariant, but a *max*-heap (negated) of size k , and the comparison key is a derived value (squared distance) rather than the raw element.

3. Heap over a derived ordering (05, 06)

Sometimes what's tracked in the heap isn't "the elements" directly, but a fact *about* them that changes as the algorithm runs.

- `meeting_rooms_ii`: the heap holds end times of rooms currently in use. A new meeting either reuses the room whose end time has already passed (root of the heap), or needs a brand new room — the heap's final size is the peak concurrent room count.
- `task_scheduler`: the heap holds remaining counts per task type, always running the most-frequent-remaining task next (greedy). A side FIFO queue holds tasks temporarily "on cooldown" and returns them to the heap once their wait time has elapsed — this is the first problem in the folder combining a heap with a second data structure to enforce a constraint the heap alone can't express.

4. K-way merge (07)

`merge_k_sorted_lists` generalizes `04_linked_list/02_merge_two_sorted_lists` from 2 lists to k . Comparing all k current heads by hand at every step would cost $O(k)$ per node; keeping them in a heap instead drops that to $O(\log k)$ per node, since the heap always surfaces the smallest of the k candidates directly. The `(val, index, node)` tuple trick — including a tiebreaker index — matters here specifically because `ListNode` objects aren't comparable, and ties on `val` would otherwise crash the comparison.

5. Heap-driven generation, not extraction (08)

`ugly_number_ii` uses a heap to *generate* a sequence in sorted order rather than to extract top/bottom elements from an existing collection. Each popped value spawns new candidates ($\times 2$, $\times 3$, $\times 5$); a `seen` set prevents the same candidate being queued twice from different paths (e.g. reaching 6 via both 2×3 and 3×2).

6. Greedy placement with a one-step delay (09)

`reorganize_string` greedily places the most frequent remaining character, using the same max-heap-by-count shape as 06. The new idea is the one-step delay: a just-placed character can't be pushed back into contention immediately, or it could get placed twice in a row. Holding it in a single `prev` variable and pushing it back only after the *next* character is chosen is a minimal way to enforce a "not immediately again" constraint without extra bookkeeping.

7. Two heaps for an always-current statistic (10)

`find_median_from_data_stream` is the hardest problem here: split the stream into a max-heap of the lower half and a min-heap of the upper half, size-balanced to within one element. The median then sits at the boundary between the two heaps — an $O(1)$ read at any point in the stream, no sorting or scanning needed, with each insert costing $O(\log n)$ to maintain the split.

Complexity summary

#	Problem	Time	Extra space
01	<code>last_stone_weight</code>	$O(n \log n)$	$O(n)$
02	<code>kth_largest_element_in_array</code>	$O(n \log k)$	$O(k)$
03	<code>kth_largest_element_in_a_stream</code>	$O(\log k)$ per add	$O(k)$
04	<code>k_closest_points_to_origin</code>	$O(n \log k)$	$O(k)$
05	<code>meeting_rooms_ii</code>	$O(n \log n)$	$O(n)$
06	<code>task_scheduler</code>	$O(n \log 26) \approx O(n)$	$O(1)$ (26 task types)
07	<code>merge_k_sorted_lists</code>	$O(n \log k)$	$O(k)$
08	<code>ugly_number_ii</code>	$O(n \log n)$	$O(n)$
09	<code>reorganize_string</code>	$O(n \log 26) \approx O(n)$	$O(n)$
10	<code>find_median_from_data_stream</code>	$O(\log n)$ per add, $O(1)$ per query	$O(n)$

Common interview follow-ups to be ready for

- `kth_largest_element_in_array`: "can you do better than $O(n \log k)$?" — yes, Quickselect averages $O(n)$, though worst-case $O(n^2)$; know the heap version cold first, then mention Quickselect as the follow-up optimization.
- `k_closest_points_to_origin`: "why squared distance instead of actual distance?" — sqrt is monotonic, so it never changes relative ordering; skipping it avoids float precision and cost for free.
- `meeting_rooms_ii`: "what if you also needed to know *which* room each meeting used?" — store room identifiers alongside end times in the heap instead of bare integers.
- `task_scheduler`: "is there a closed-form formula instead of simulation?" — yes (based on the max count and how many tasks tie for it), but the simulation shown here is easier to derive live and to explain the reasoning behind, which interviewers often prefer.
- `merge_k_sorted_lists`: "what if k is very large, larger than the total number of nodes?" — the heap never holds more than k elements at once regardless of total node count, so this doesn't change the complexity; be ready to state that explicitly.
- `find_median_from_data_stream`: "what if numbers can also be *removed* from the stream?" — plain heaps don't support efficient arbitrary removal; this needs lazy deletion (mark-and-skip) or a balanced BST/order-statistics structure instead — good to mention as the natural harder follow-up.

Self-check before moving on

You should be able to, without looking at the code:

- Explain the negate-for-max-heap trick and why Python's `heapq` only offers a min-heap.
- State the size- k heap invariant from memory and explain why the root is always the "worst of the best k ."
- Explain why `merge_k_sorted_lists` needs a tiebreaker in its heap tuples.
- Walk through the two-heap balancing steps in `find_median_from_data_stream` on a small example (5-6 numbers) by hand, tracking both heaps after each insert.

If any of those aren't fluent yet, this topic isn't "done" per the README's definition — revisit before moving to `09_tries`.

Programs

01. Last Stone Weight

01_last_stone_weight.py

Explanation

Problem: You're given stone weights. Repeatedly smash the two heaviest stones together: if equal, both are destroyed; otherwise the lighter is destroyed and the heavier becomes a new stone with weight (heavy - light). Return the weight of the last stone left, or 0 if none remain.

Source : LeetCode 1046 - Last Stone Weight

Example:

Input: [2, 7, 4, 1, 8, 1]

Output: 1

Idea: "repeatedly grab the two largest" is exactly what a max-heap is for. Python's heapq only implements a *min*-heap, so the standard trick is to push negated values - the smallest negated value is the largest original value, which is what heappop then returns. Simulate directly: pop the two largest, and if they weren't equal, push the difference back in as a new stone.

Code

```
import heapq

def last_stone_weight(stones: list[int]) -> int:
    heap = [-s for s in stones]
    heapq.heapify(heap)

    while len(heap) > 1:
        first = -heapq.heappop(heap)
        second = -heapq.heappop(heap)
        if first != second:
            heapq.heappush(heap, -(first - second))

    return -heap[0] if heap else 0

if __name__ == "__main__":
    tests = [
        ([2, 7, 4, 1, 8, 1], 1),
        ([1], 1),
        ([], 0),
        ([2, 2], 0),
    ]

    for i, (inp, expected) in enumerate(tests, 1):
        got = last_stone_weight(inp[:])
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")
```

02. Kth Largest Element In Array

02_kth_largest_element_in_array.py

Explanation

Problem: Given an integer array `nums` and integer `k`, return the `k`th largest element (`k`th largest, not `k`th distinct).

Source : LeetCode 215 - Kth Largest Element in an Array

Example:

Input: `nums = [3, 2, 1, 5, 6, 4]`, `k = 2`

Output: 5

Idea: sorting the whole array is $O(n \log n)$. Instead, maintain a *min*-heap of exactly size `k` containing the `k` largest values seen so far - its smallest element (the heap's root) is then, by definition, the `k`th largest overall. Seed the heap with the first `k` elements, then for every remaining element: if it's bigger than the heap's smallest, it belongs in the top-`k` and should replace that smallest element (`heapreplace` does `pop+push` in one $O(\log k)$ operation). Total cost $O(n \log k)$, better than a full sort when `k` is small.

Code

```
import heapq

def find_kth_largest(nums: list[int], k: int) -> int:
    heap = nums[:k]
    heapq.heapify(heap)

    for n in nums[k:]:
        if n > heap[0]:
            heapq.heapreplace(heap, n)

    return heap[0]

if __name__ == "__main__":
    tests = [
        ([3, 2, 1, 5, 6, 4], 2, 5),
        ([3, 2, 3, 1, 2, 4, 5, 5, 6], 4, 4),
        ([1], 1, 1),
    ]

    for i, (nums, k, expected) in enumerate(tests, 1):
        got = find_kth_largest(nums, k)
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")
```

03. Kth Largest Element In A Stream

03_kth_largest_element_in_a_stream.py

Explanation

Problem: Design a class that, given an initial array and integer k, supports add(val) - adding a new value to the stream and returning the kth largest element among all values added so far (including the initial array).

Source : LeetCode 703 - Kth Largest Element in a Stream

Example:

```
kth = KthLargest(3, [4, 5, 8, 2])
```

```
kth.add(3) -> 4
```

```
kth.add(5) -> 5
```

```
kth.add(10) -> 5
```

```
kth.add(9) -> 8
```

```
kth.add(4) -> 8
```

Idea: the streaming version of 02 - same size-k min-heap invariant (heap always holds exactly the k largest values seen so far, root is the kth largest), but now the heap needs to *persist* across calls instead of being built once. Every add() just pushes the new value and, if that grows the heap past size k, pops the smallest - $O(\log k)$ per call, rather than re-running a fresh $O(n \log k)$ pass over the whole history each time.

Code

```
import heapq

class KthLargest:
    def __init__(self, k: int, nums: list[int]):
        self.k = k
        self.heap = nums[:]
        heapq.heapify(self.heap)
        while len(self.heap) > k:
            heapq.heappop(self.heap)

    def add(self, val: int) -> int:
        heapq.heappush(self.heap, val)
        if len(self.heap) > self.k:
            heapq.heappop(self.heap)
        return self.heap[0]

if __name__ == "__main__":
    kth = KthLargest(3, [4, 5, 8, 2])
    ops = [
        (kth.add, (3, ), 4),
        (kth.add, (5, ), 5),
        (kth.add, (10, ), 5),
        (kth.add, (9, ), 8),
        (kth.add, (4, ), 8),
    ]

    for i, (fn, args, expected) in enumerate(ops, 1):
        got = fn(*args)
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (add{args} -> got {got}, expected {expected})")
```

04. K Closest Points To Origin

04_k_closest_points_to_origin.py

Explanation

Problem: Given an array of points on the 2D plane and an integer k , return the k points closest to the origin $(0, 0)$. Any order is acceptable.

Source : LeetCode 973 - K Closest Points to Origin

Example:

Input: points = $[[1, 3], [-2, 2]]$, $k = 1$

Output: $[-2, 2]$

Idea: same size- k heap invariant as 02/03, applied to a derived key (squared Euclidean distance - no need for an actual sqrt, since it's monotonic and comparisons care only about relative order) instead of the raw value. This time it's a *max*-heap of size k holding the k closest points found so far, so the point that's furthest among those k (the one to evict if something closer shows up) sits at the root - achieved via the same negate-for-max-heap trick as 01.

Code

```
import heapq

def k_closest(points: list[list[int]], k: int) -> list[list[int]]:
    heap = []

    for x, y in points:
        dist_sq = x * x + y * y
        heapq.heappush(heap, (-dist_sq, x, y))
        if len(heap) > k:
            heapq.heappop(heap)

    return [[x, y] for _, x, y in heap]

if __name__ == "__main__":
    tests = [
        ([[1, 3], [-2, 2]], 1, [[-2, 2]]),
        ([[3, 3], [5, -1], [-2, 4]], 2, [[3, 3], [-2, 4]]),
        ([[0, 1]], 1, [[0, 1]]),
    ]

    for i, (points, k, expected) in enumerate(tests, 1):
        got = k_closest(points, k)
        got_set = sorted(map(tuple, got))
        expected_set = sorted(map(tuple, expected))
        status = "PASS" if got_set == expected_set else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")
```

05. Meeting Rooms II

05_meeting_rooms_ii.py

Explanation

Problem: Given an array of meeting time intervals [start, end], return the minimum number of conference rooms required to hold all of them (overlapping meetings need separate rooms).

Source : LeetCode 253 - Meeting Rooms II

Example:

Input: [[0, 30], [5, 10], [15, 20]]

Output: 2

Idea: sort meetings by start time, then process them in that order while tracking the *end times of all currently-ongoing meetings* in a min-heap. For each new meeting, check the heap's smallest end time (the room that frees up soonest): if that room is already free by the time this meeting starts, reuse it (pop the old end time, push the new one - same room, new occupant). Otherwise, no existing room is free yet, so a new one is needed (just push, growing the heap). The heap's final size is exactly the peak number of simultaneously occupied rooms - the answer.

Code

```
import heapq

def min_meeting_rooms(intervals: list[list[int]]) -> int:
    if not intervals:
        return 0

    intervals = sorted(intervals)
    heap: list[int] = [] # end times of ongoing meetings

    for start, end in intervals:
        if heap and heap[0] <= start:
            heapq.heapreplace(heap, end)
        else:
            heapq.heappush(heap, end)

    return len(heap)

if __name__ == "__main__":
    tests = [
        ([0, 30], [5, 10], [15, 20]), 2,
        ([7, 10], [2, 4]), 1,
        ([1, 5], [8, 9], [8, 9]), 2,
        ([], 0),
    ]

    for i, (inp, expected) in enumerate(tests, 1):
        got = min_meeting_rooms(inp)
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")
```

06. Task Scheduler

06_task_scheduler.py

Explanation

Problem: Given a list of tasks (letters) and a cooldown n (the same task must wait at least n intervals before running again), return the minimum total number of intervals (including idle slots) needed to finish all tasks. Each interval runs one task or sits idle.

Source : LeetCode 621 - Task Scheduler

Example:

Input: tasks = ["A","A","A","B","B","B"], n = 2

Output: 8 (A B _ A B _ A B - the two idle slots are unavoidable)

Idea: greedily run whichever remaining task has the *highest* count first - a max-heap of task counts, same negate trick as 01. After running a task, it can't run again for n intervals, so it goes into a side "cooldown queue" holding (time_it_becomes_available_again, remaining_count) instead of straight back into the heap. Only once the clock reaches that available time does it get pushed back onto the heap to compete again. The simulation naturally produces idle intervals: if the heap is empty but the cooldown queue isn't, time still has to advance with nothing to run.

Code

```
import heapq
from collections import Counter, deque

def least_interval(tasks: list[str], n: int) -> int:
    counts = Counter(tasks)
    max_heap = [-c for c in counts.values()]
    heapq.heapify(max_heap)

    time = 0
    cooldown: deque[tuple[int, int]] = deque() # (time_available_again, remaining_count)

    while max_heap or cooldown:
        time += 1
        if max_heap:
            remaining = 1 + heapq.heappop(max_heap) # remaining is <= 0 (negated)
            if remaining:
                cooldown.append((time + n, remaining))
            if cooldown and cooldown[0][0] == time:
                heapq.heappush(max_heap, cooldown.popleft()[1])

    return time

if __name__ == "__main__":
    tests = [
        (["A", "A", "A", "B", "B", "B"], 2, 8),
        (["A", "A", "A", "B", "B", "B"], 0, 6),
        (["A", "A", "A", "A", "A", "A", "B", "C", "D", "E", "F", "G"], 2, 16),
    ]

    for i, (tasks, n, expected) in enumerate(tests, 1):
        got = least_interval(tasks, n)
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")
```

07. Merge K Sorted Lists

07_merge_k_sorted_lists.py

Explanation

Problem: Given k sorted linked lists, merge them into one sorted linked list.

Source : LeetCode 23 - Merge k Sorted Lists

Example:

Input: `[[1,4,5], [1,3,4], [2,6]]`

Output: `[1,1,2,3,4,4,5,6]`

Idea: 04_linked_list/02_merge_two_sorted_lists merges **two** lists by always taking whichever head is smaller. Merging k lists the same way pairwise would cost $O(kn)$ per level. Instead, keep the current head of every list in a min-heap at once - the heap root is always the smallest value across **all** k lists, so popping it and pushing its successor keeps the invariant with $O(\log k)$ work per node instead of $O(k)$ work per node (comparing all k heads by hand). A tuple (val, index, node) is pushed rather than the bare node, because ListNode isn't comparable and two nodes could tie on value - the unique **index** (which list it came from) breaks ties before Python would ever try to compare two ListNode objects directly.

Code

```
import heapq

class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

def to_linked_list(values):
    dummy = ListNode()
    curr = dummy
    for v in values:
        curr.next = ListNode(v)
        curr = curr.next
    return dummy.next

def to_python_list(head):
    result = []
    while head:
        result.append(head.val)
        head = head.next
    return result

def merge_k_lists(lists: list) -> ListNode:
    heap = []
    for i, node in enumerate(lists):
        if node:
            heapq.heappush(heap, (node.val, i, node))
```

```

dummy = ListNode()
tail = dummy

while heap:
    val, i, node = heapq.heappop(heap)
    tail.next = node
    tail = tail.next
    if node.next:
        heapq.heappush(heap, (node.next.val, i, node.next))

return dummy.next

if __name__ == "__main__":
    tests = [
        ([1, 4, 5], [1, 3, 4], [2, 6]), [1, 1, 2, 3, 4, 4, 5, 6]),
        ([], []),
        ([[]], []),
        ([[1]], [1]),
    ]

    for i, (lists_vals, expected) in enumerate(tests, 1):
        lists = [to_linked_list(v) for v in lists_vals]
        got = to_python_list(merge_k_lists(lists))
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")

```

08. Ugly Number II

08_ugly_number_ii.py

Explanation

Problem: An ugly number is a positive integer whose only prime factors are 2, 3, and 5. Given n, return the nth ugly number (1 is conventionally the first).

Source : LeetCode 264 - Ugly Number II

Example:

Input: n = 10

Output: 12 (sequence: 1, 2, 3, 4, 5, 6, 8, 9, 10, 12)

Idea: generate ugly numbers in sorted order using a min-heap instead of testing every integer for ugliness (which would need factoring each candidate). Start the heap with 1. Repeatedly pop the smallest ugly number found so far, and push its three "descendants" (popped * 2, popped * 3, popped * 5) - each of which is also necessarily ugly, since it's a product of an ugly number and one of the three allowed primes. A **seen** set prevents the same value being pushed multiple times (e.g. 2*3 and 3*2 would otherwise both reach 6). Popping n times gives exactly the nth ugly number.

Code

```

import heapq

def nth_ugly_number(n: int) -> int:

```

```

heap = [1]
seen = {1}
ugly = 1

for _ in range(n):
    ugly = heapq.heappop(heap)
    for factor in (2, 3, 5):
        candidate = ugly * factor
        if candidate not in seen:
            seen.add(candidate)
            heapq.heappush(heap, candidate)

return ugly

if __name__ == "__main__":
    tests = [
        (10, 12),
        (1, 1),
        (15, 24),
    ]

    for i, (n, expected) in enumerate(tests, 1):
        got = nth_ugly_number(n)
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")

```

09. Reorganize String

09_reorganize_string.py

Explanation

Problem: Given a string *s*, rearrange its characters so that no two adjacent characters are the same. Return any valid rearrangement, or "" if impossible.

Source : LeetCode 767 - Reorganize String

Example:

Input: "aab"

Output: "aba" (any valid rearrangement is accepted)

Idea: greedily place the *most frequent remaining* character at each position - a max-heap of (count, char), same negate trick as 01/06.

The subtlety is avoiding placing the same character twice in a row:

after popping and placing a character, don't push it straight back

(it might get popped again immediately if it's still the most

frequent) - instead hold it in *prev* for exactly one step, and push

it back only after the *next* character has been placed. This

guarantees at least one different character separates two uses of

the same letter. Impossible cases are detected up front: if the most

frequent character's count exceeds $(\text{len}(s) + 1) // 2$, it can't be

spread out enough no matter the arrangement.

Code

```
import heapq
```

```

from collections import Counter

def reorganize_string(s: str) -> str:
    counts = Counter(s)
    max_heap = [(-c, ch) for ch, c in counts.items()]
    heapq.heapify(max_heap)

    if max_heap and -max_heap[0][0] > (len(s) + 1) // 2:
        return ""

    result = []
    prev = None # (count, char) of the previously placed character

    while max_heap:
        count, ch = heapq.heappop(max_heap)
        result.append(ch)
        if prev and prev[0] < 0:
            heapq.heappush(max_heap, prev)
        count += 1 # count is negative; +1 means "used one instance"
        prev = (count, ch)

    return "".join(result)

def _is_valid_reorganization(original: str, result: str) -> bool:
    if sorted(original) != sorted(result):
        return False
    return all(result[i] != result[i + 1] for i in range(len(result) - 1))

if __name__ == "__main__":
    tests = ["aab", "aaab", "vvvlo", "a", ""]
    expected_possible = [True, False, True, True, True]

    for i, (s, possible) in enumerate(zip(tests, expected_possible), 1):
        got = reorganize_string(s)
        if not possible:
            status = "PASS" if got == "" else "FAIL"
        else:
            status = "PASS" if _is_valid_reorganization(s, got) else "FAIL"
        print(f"Test {i}: {status} (input={s!r}, got {got!r})")

```

10. Find Median From Data Stream

10_find_median_from_data_stream.py

Explanation

Problem: Design a structure that supports `add_num(val)`, adding an integer to a running stream, and `find_median()`, returning the median of all numbers added so far, at any point in the stream.

Source : LeetCode 295 - Find Median from Data Stream

Example:

```

mf = MedianFinder()
mf.add_num(1); mf.add_num(2)
mf.find_median() -> 1.5
mf.add_num(3)

```

mf.find_median() -> 2.0

Idea: the hardest problem in this folder, and the canonical "two heaps" pattern. Split all numbers seen so far into two halves: **small**, a *max*-heap (negated) holding the smaller half, and **large**, a *min*-heap holding the bigger half - kept balanced so their sizes differ by at most one. The median is then always sitting at the top of one or both heaps, an $O(1)$ lookup, with no need to ever sort or scan the full stream. Every `add_num` does three heap ops to maintain two invariants:

1. every value in **small** $\leq$ every value in **large** (enforced by always pushing into **small** first, then immediately moving its largest into **large** - guarantees correct ordering even if the new value belonged in **large**).
2. $\text{len}(\text{large}) \leq \text{len}(\text{small}) \leq \text{len}(\text{large}) + 1$ (enforced by moving one value back from **large** to **small** if **large** grew too big).

Code

```
import heapq

class MedianFinder:
    def __init__(self):
        self.small: list[int] = [] # max-heap (negated), holds the lower half
        self.large: list[int] = [] # min-heap, holds the upper half

    def add_num(self, num: int) -> None:
        heapq.heappush(self.small, -num)
        heapq.heappush(self.large, -heapq.heappop(self.small))

        if len(self.large) > len(self.small):
            heapq.heappush(self.small, -heapq.heappop(self.large))

    def find_median(self) -> float:
        if len(self.small) > len(self.large):
            return float(-self.small[0])
        return (-self.small[0] + self.large[0]) / 2

if __name__ == "__main__":
    mf = MedianFinder()
    ops = [
        (mf.add_num, (1,), None),
        (mf.add_num, (2,), None),
        (mf.find_median, (), 1.5),
        (mf.add_num, (3,), None),
        (mf.find_median, (), 2.0),
        (mf.add_num, (10,), None),
        (mf.find_median, (), 2.5),
        (mf.add_num, (-5,), None),
        (mf.find_median, (), 2.0),
    ]

    for i, (fn, args, expected) in enumerate(ops, 1):
        got = fn(*args)
        if expected is None:
            status = "PASS"
```

```
else:  
    status = "PASS" if got == expected else "FAIL"  
    print(f"Test {i}: {status} ({fn.__name__}{args} -> got {got}, expected {expected})")
```